

团结引擎游戏开发规范文档

适用对象：Unity 开发团队、技术负责人、程序、TA、策划协作人员

目标：提升代码质量、项目可维护性、性能表现与团队协作效率

最后更新：2025 年 12 月 17 日

零、代码架构要求以及插件建议

0.1 架构相关

- 有清晰的流程控制结构
- 有统一的Log管理功能，支持一键开启或者关闭
- 配置功能建议使用Luban > <https://www.databale.cn/>
- 资源构建管理建议使用YooAsset > <https://www.yooasset.com/>

0.2 插件相关

- 动效表现推荐使用DOTween或DOTweenPro
- 对象池建议使用PathologicalGames的PoolManager
- 本地数据管理建议使用EasySave

0.3 其他

- 团结引擎版本建议使用1.7及以上版本
- 3D项目使用URP管线

一、项目结构规范

1.1 文件夹结构

建议采用以下标准化目录结构，确保资源分类清晰、职责明确：

Plain Text

Assets/

— Art/	# 美术资源
— Characters/	
— Environments/	
— UI/	
— VFX/	
— Audio/	# 音频资源
— Music/	
— SFX/	
— Prefabs/	# 预制体 (按功能或模块分类)
— Characters/	
— UI/	
— Props/	
— Scenes/	# 场景文件
— Main/	# 正式场景
— Test/	# 测试/调试场景
— Scripts/	# C# 脚本代码
— Core/	# 核心系统 (GameManager、EventSystem 等)
— Gameplay/	# 游戏逻辑 (Player、Enemy、Combat 等)
— UI/	# 用户界面逻辑
— Utilities/	# 工具类 (Extensions、Helpers、Singleton 等)
— ThirdParty/	# 第三方插件 (如 DOTween、TextMeshPro)
— Resources/	# 仅用于 Resources.Load() 的资源 (慎用)
— StreamingAssets/	# 原始文件 (如 JSON、视频等, 打包后保留原始格式)
— Editor/	# 自定义编辑器脚本 (必须位于此目录)
— Plugins/	# 原生插件 (Android/iOS 等平台专用)

1.2 命名约定

- **文件夹命名**：使用 PascalCase (如 Gameplay, UIElements)
- **资源文件命名**：使用 PascalCase (如 PlayerController.prefab, MainMenu.unity)
- **禁止使用**：空格、中文、特殊符号 (如 #, %, &)
- **场景命名**：以功能命名, 如 Level_01.unity, MainMenu.unity

二、脚本编写规范 (C#)

2.1 类与文件命名

- 类名必须与文件名完全一致 (Unity 强制要求)
- 使用 PascalCase (如 `PlayerHealth.cs`)
- 接口以 `I` 开头 (如 `IDamageable.cs`)
- 抽象基类可加 `Base` 后缀 (如 `EnemyBase.cs`)

2.2 代码风格

- 缩进：使用 Tab
- 大括号风格：K&R 风格 (左大括号换行)

```
C#
if (condition)
{
    DoSomething();
}
```

- 访问修饰符：必须显式声明 (`private`、`public`、`protected`)
- 成员变量命名：
 - 私有字段：`_camelCase` (如 `_playerSpeed`)
 - 公共属性/字段：`PascalCase` (如 `MaxHealth`)
 - 静态常量：`PascalCase` (如 `MAX_ENEMY_COUNT`)

2.3 MonoBehaviour 使用规范

- 避免在 `Update()` 中执行昂贵操作 (如 `Find()`、`GetComponent()`)
- 组件引用应在 `Awake()` 或 `Start()` 中缓存

```
C#
private Rigidbody _rb;
private void Awake()
{
    _rb = GetComponent<Rigidbody>();
}
```

- 优先使用 `TryGetComponent<T>(out T component)` 而非 `GetComponent<T>() != null`

- 避免在 `OnDestroy()` 中调用其他 `GameObject` 的方法 (可能已销毁)

2.4 性能与安全

- 避免在循环中分配堆内存 (如 `new Vector3`, `new List<T>`)
- 使用对象池管理高频创建/销毁对象 (如子弹、特效)
- 使用 `[SerializeField]` 曝光私有字段供 Inspector 编辑 , 而非设为 `public`
- 禁止使用 `FindObjectOfType<T>()` 和 `Resources.Load()` (性能差、耦合高、难以维护)

三、场景与对象管理

3.1 GameObject 命名

- 使用语义清晰的名称 (如 `Player`, `Enemy_Skeleton`, `UI_MainMenu`)
- 禁止使用默认名称 (如 `GameObject (1)`)
- 层级嵌套深度建议 ≤ 5 层 , 避免“俄罗斯套娃”式结构

3.2 Tag 与 Layer

- **Tag** : 用于逻辑识别 (如 `"Player"`, `"Enemy"`, `"Pickup"`)
- **Layer** : 用于物理/渲染过滤 (如 `"IgnoreRaycast"`, `"UI"`, `"Water"`)
- 自定义 Tag/Layer 需在项目初期统一规划 , 避免冲突

四、资源管理规范

4.1 资源导入设置

资源类型	推荐设置
纹理	关闭未使用的 Mip Maps ; UI 用 ETC2 , 纹理贴图用 ASTC ; 合理设置 Max Size , 不超过1024

模型	使用 FBX 格式，控制面数以及动作的骨骼数
音频	背景音乐：Compressed In Memory + Vorbis；短音效：Decompress On Load

4.2 动作

- 状态命名清晰（如 Idle, Run, Attack_Light）
- 动作可以使用Animancer插件

五、UI 开发规范

5.1

- 清晰的MVC结构
- 合理的UI界面层级管理
- 合理的图集策略
- 动静Canvas分离

六、发布

6.1

- 安卓包名需要统一的前缀com.youdoo，需要严格遵守
- 发布时使用 IL2CPP 脚本后端（而非 Mono）
- 发布平台选择ARM64
- Android：API Level 30

七、多语言

1) 游戏中涉及文字展示的文本信息，需归纳在配置表Language.xlsx中，禁止

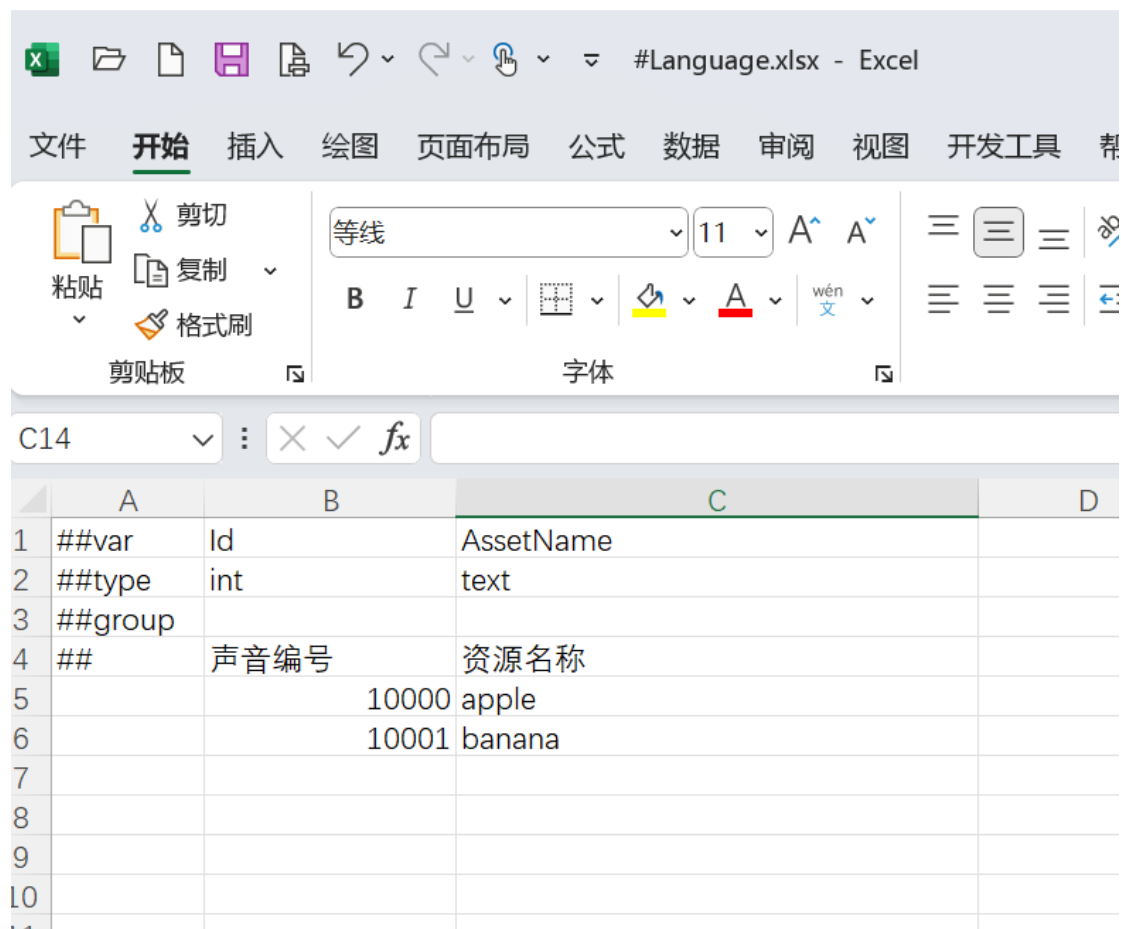
使用硬编码;

2) 游戏中涉及文字展示的图片信息, 需统一放置在... \lang\的目录中, 命名规范为xxx_语言_编号;

3) 多语言规则必须严格遵守, 开发包需要提供简体中文、繁体中文和英语三个版本的资源文件;

Luban多语言使用示例

1、配置表



	A	B	C	D
1	##var	Id	AssetName	
2	##type	int	text	
3	##group			
4	##	声音编号	资源名称	
5		10000	apple	
6		10001	banana	
7				
8				
9				
10				

2、texts.json配置



```
{
  "key": "apple", "en": "apple name", "zh": "苹果 name",
  "key": "banana", "en": "banana name", "zh": "香蕉 name"
}
```

3、gen.bat

```
D:\UnityFiles\sw16_yd_base\Excel\gen.bat - Notepad++
文件(F) 编辑(E) 搜索(S) 视图(V) 编码(N) 语言(L) 设置(T) 工具(O) 宏(M) 运行(R) 插件(P) 窗口(W) ?
新文件 8 新文件 1 gen.bat gen.bat log.txt
1 set WORKSPACE=..
2 set PROJECT_NAME=yd_base
3 set LUBAN_DLL=%WORKSPACE%\Excel\Tool\Luban.dll
4 set CONF_ROOT=%WORKSPACE%\Excel
5 set OUTPUT_DIR=%WORKSPACE%\UnityProject\%PROJECT_NAME%\Assets\GameMain\DataTables
6
7 dotnet %LUBAN_DLL% ^
8     -t client ^
9     -c cs-simple-json ^
10    -d json ^
11    --conf %CONF_ROOT%\luban.conf ^
12    -x outputDataDir=%OUTPUT_DIR%\outputDataDir\json\zh ^
13    -x outputCodeDir=%OUTPUT_DIR%\outputCodeDir ^
14    -x l10n.provider=default ^
15    -x l10n.textFile.path=%WORKSPACE%\Excel\Datas\l10n\texts.json ^
16    -x l10n.textFile.keyFieldName=key ^
17    -x l10n.textFile.languageFieldName=zh ^
18    -x l10n.convertTextKeyToValue=1
19
20 dotnet %LUBAN_DLL% ^
21     -t client ^
22     -c cs-bin ^
23     -d bin ^
24     --conf %CONF_ROOT%\luban.conf ^
25     -x outputDataDir=%WORKSPACE%\UnityProject\%PROJECT_NAME%\Assets\StreamingAssets\zh ^
26     -x outputDataDir2=%OUTPUT_DIR%\outputDataDir\byte\zh ^
27     -x outputCodeDir=%OUTPUT_DIR%\outputCodeDir ^
28     -x l10n.provider=default ^
29     -x l10n.textFile.path=%WORKSPACE%\Excel\Datas\l10n\texts.json ^
30     -x l10n.textFile.keyFieldName=key ^
31     -x l10n.textFile.languageFieldName=zh ^
32     -x l10n.convertTextKeyToValue=1
33
34 dotnet %LUBAN_DLL% ^
35     -t client ^
36     -c cs-simple-json ^
37     -d json ^
38     --conf %CONF_ROOT%\luban.conf ^
39     -x outputDataDir=%OUTPUT_DIR%\outputDataDir\json\en ^
40     -x outputCodeDir=%OUTPUT_DIR%\outputCodeDir ^
41     -x l10n.provider=default ^
42     -x l10n.textFile.path=%WORKSPACE%\Excel\Datas\l10n\texts.json ^
43     -x l10n.textFile.keyFieldName=key ^
44     -x l10n.textFile.languageFieldName=en ^
45     -x l10n.convertTextKeyToValue=1
46
47 dotnet %LUBAN_DLL% ^
48     -t client ^
49     -c cs-bin ^
50     -d bin ^
51     --conf %CONF_ROOT%\luban.conf ^
52     -x outputDataDir=%WORKSPACE%\UnityProject\%PROJECT_NAME%\Assets\StreamingAssets\en ^
53     -x outputDataDir2=%OUTPUT_DIR%\outputDataDir\byte\en ^
54     -x outputCodeDir=%OUTPUT_DIR%\outputCodeDir ^
55     -x l10n.provider=default ^
56     -x l10n.textFile.path=%WORKSPACE%\Excel\Datas\l10n\texts.json ^
57     -x l10n.textFile.keyFieldName=key ^
58     -x l10n.textFile.languageFieldName=en ^
59     -x l10n.convertTextKeyToValue=1
60
61 set BYTE_FOLDER=%OUTPUT_DIR%\outputDataDir\byte
62 if exist "%BYTE_FOLDER%\zh" (
63     del /q "%BYTE_FOLDER%\zh\*.bytes"
64 )
65 if exist "%BYTE_FOLDER%\en" (
66     del /q "%BYTE_FOLDER%\en\*.bytes"
67 )
68
69 if not exist "%BYTE_FOLDER%\zh" mkdir "%BYTE_FOLDER%\zh"
70 if not exist "%BYTE_FOLDER%\en" mkdir "%BYTE_FOLDER%\en"
71
72 xcopy "%WORKSPACE%\UnityProject\%PROJECT_NAME%\Assets\StreamingAssets\zh\*.bytes" "%BYTE_FOLDER%\zh\" /Y /I
73 xcopy "%WORKSPACE%\UnityProject\%PROJECT_NAME%\Assets\StreamingAssets\en\*.bytes" "%BYTE_FOLDER%\en\" /Y /I
74
75 pause
```

4、结果

```
tyFiles\sw16_yd_base\UnityProject\yd_base\Assets\GameMain\DataTables\outputDataDir\json\tblanguage.json - Notepad++
编辑(E) 搜索(S) 视图(V) 编码(N) 语言(L) 设置(T) 工具(O) 宏(M) 运行(R) 插件(P) 窗口(W) ?
[新文件 1] [texts.json] [gen.bat] [tblanguage.json] [tblanguage.json]
[
  {
    "Id": 10000,
    "AssetName": "苹果 name"
  },
  {
    "Id": 10001,
    "AssetName": "香蕉 name"
  }
]
```

```
UnityFiles\sw16_yd_base\UnityProject\yd_base\Assets\GameMain\DataTables\outputDataDir\json\en\tblanguage.json - Notepad++
) 编辑(E) 搜索(S) 视图(V) 编码(N) 语言(L) 设置(T) 工具(O) 宏(M) 运行(R) 插件(P) 窗口(W) ?
[文件 8] [新文件 1] [texts.json] [gen.bat] [tblanguage.json] [tblanguage.json]
[
  {
    "Id": 10000,
    "AssetName": "apple name"
  },
  {
    "Id": 10001,
    "AssetName": "banana name"
  }
]
```

这是静态多语言的使用方式，根据需要导出对应语言的配置，如果有其他语言，直接修改texts.json在后面新增字段填内容就可以，策划配置多语言的时候直接修改texts.json，其他配表逻辑一致。

这里的gen.bat写在一起了，可以根据en，zh等拆分成多个bat，这样会更清晰。

另，如果觉得texts.json不方便操作可以处理成一个excel表格，使用之前单独转成json。